EXPERIMENT 6

AIM

To visualize and analyze the given dataset using various plots such as bar plots, count plots, histograms, matrix plots etc.. with the help of the Seaborn library, for better interpretation and understanding of trends and patterns.

PREREQUISITES & REQUIREMENTS:

1. Computer with Python Installed
2. A dataset for Practice
3. Jupytor Notebook/Google Colab
4. Knowledge on Pandas, Matplotlib & Seaborn Libraries

SEABORN

1. Seaborn is a Statistical library designed to interact with Pandas Dataframes.
2. It is used to create common statistical plot types.
3. It uses a simpler one-line syntax
4. Since, seaborn is built with matplotlib, so still we use plt method to create plots.
5. Choosing the type of plot depends on the questions and relations we are working with.
6. For more useful info about seaborn - https://seaborn.pydata.org/
7. While working with seaborn it should be imported - import seaborn as sns

Step 1: Import the required libraries & dataset

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Read the Renewable Energy dataset
df_241UDS0096 = pd.read_csv("/content/renewable_energy_dataset.csv")

df_241UDS0096.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 8 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Year                        16 non-null     int64
 1   Solar_Capacity_GW           16 non-null     float64
 2   Wind_Capacity_GW            16 non-null     float64
 3   Hydro_Capacity_GW           16 non-null     float64
 4   Renewable_Generation_TWh    16 non-null     float64
 5   Investment_Crore            16 non-null     int64
 6   Energy_Demand_TWh           16 non-null     float64
 7   Total_Renewable_Capacity_GW  16 non-null     float64
dtypes: float64(6), int64(2)
memory usage: 1.1 KB
```

In [ ]:
```
#Display the dataset
df_241UDS0096.head()
```

	Year	Solar_Capacity_GW	Wind_Capacity_GW	Hydro_Capacity_GW	Renewable_Generation_TWh	Investment_Crore	Energy_Demand_TWh	Total_Renewable_Cap
0	2010	3.84	11.88	37.99	92.41	20844	844.51	53.71
1	2011	9.41	16.07	38.29	92.33	29880	918.93	63.77
2	2012	18.54	17.20	40.92	121.60	32015	1006.95	76.66
3	2013	27.89	19.19	40.42	131.30	42912	1017.04	87.50
4	2014	30.65	25.23	42.70	143.93	47799	1069.79	98.58

In [ ]:
```
#Check the columns
df_241UDS0096.columns
```

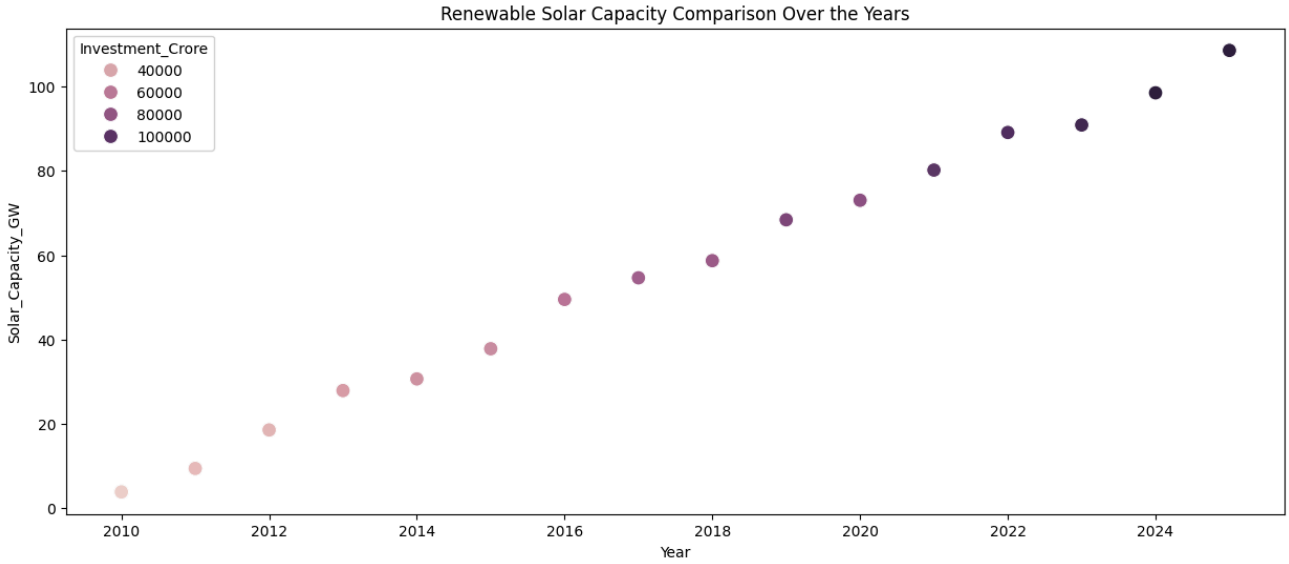
```
Index(['Year', 'Solar_Capacity_GW', 'Wind_Capacity_GW', 'Hydro_Capacity_GW',
       'Renewable_Generation_TWh', 'Investment_Crore', 'Energy_Demand_TWh',
       'Total_Renewable_Capacity_GW'],
      dtype='object')
```

Step 2: Visualize Data Using Various Plots

In [ ]:
```
# Scatter Plot
plt.figure(figsize=(15, 6))

sns.scatterplot(
    x='Year',
    y='Solar_Capacity_GW',
    hue='Investment_Crore',
    data=df_241UDS0096,
    s=100
)

plt.title('Renewable Solar Capacity Comparison Over the Years')
plt.xlabel('Year')
plt.ylabel('Solar_Capacity_GW')
plt.show()
```

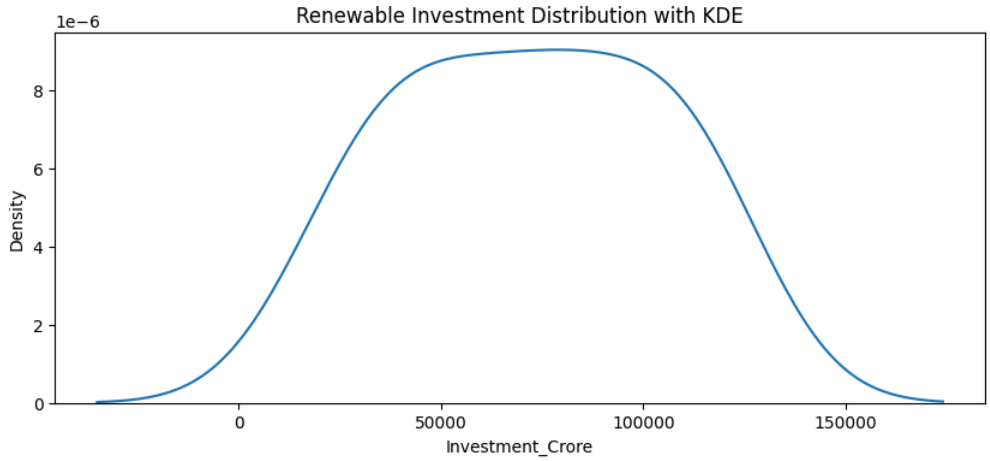


Histograms & KDE

In [ ]:
```
#KDE Plot
plt.figure(figsize=(10, 4))

sns.kdeplot(
    data=df_241UDS0096,
    x='Investment_Crore'
)

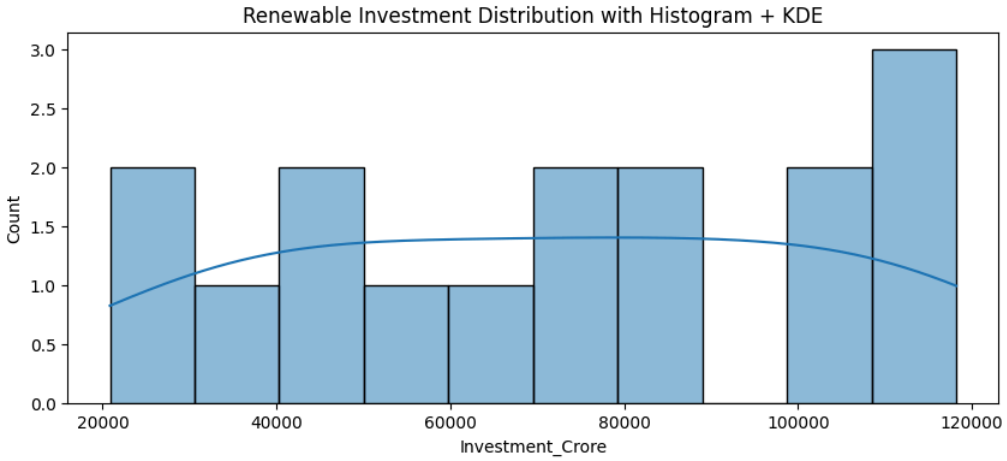
plt.title('Renewable Investment Distribution with KDE')
plt.xlabel('Investment_Crore')
plt.show()
```



In [ ]:
```
#Histogram + KDE
plt.figure(figsize=(10, 4))

sns.histplot(
    data=df_241UDS0096,
    x='Investment_Crore',
    bins=10,
    kde=True
)

plt.title('Renewable Investment Distribution with Histogram + KDE')
plt.xlabel('Investment_Crore')
plt.show()
```



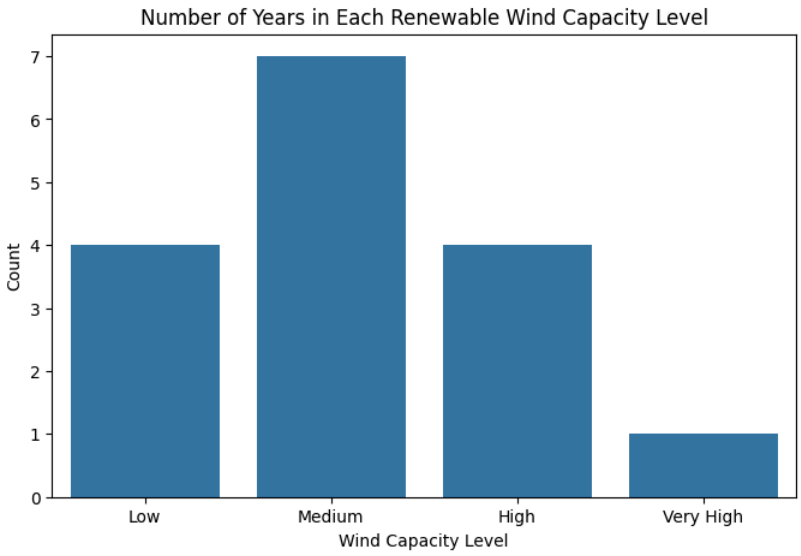
Count Plot

In [ ]:
```
df_241UDS0096['Wind_Level_241UDS0096'] = pd.cut(
    df_241UDS0096['Wind_Capacity_GW'],
    bins=[0,20,40,50,60],
    labels=['Low', 'Medium', 'High', 'Very High']
)
```

In [ ]:
```
plt.figure(figsize=(8, 5))

sns.countplot(
    x='Wind_Level_241UDS0096',
    data=df_241UDS0096
)

plt.title('Number of Years in Each Renewable Wind Capacity Level')
plt.xlabel('Wind Capacity Level')
plt.ylabel('Count')
plt.show()
```

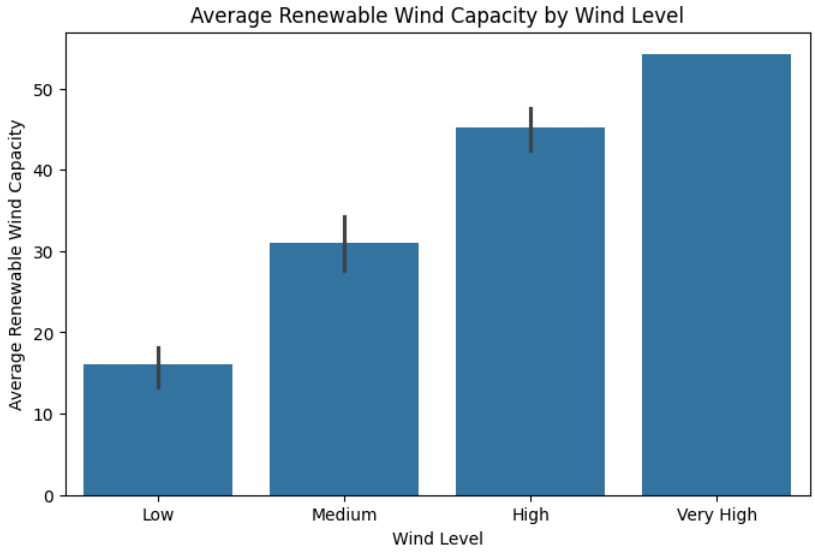


Bar Plot

In [ ]:
```
#Average Renewable Wind Capacity by Wind Level
plt.figure(figsize=(8, 5))

sns.barplot(
    x='Wind_Level_241UDS0096',
    y='Wind_Capacity_GW',
    data=df_241UDS0096,
    estimator=np.mean
)

plt.title('Average Renewable Wind Capacity by Wind Level')
plt.xlabel('Wind Level')
plt.ylabel('Average Renewable Wind Capacity')
plt.show()
```

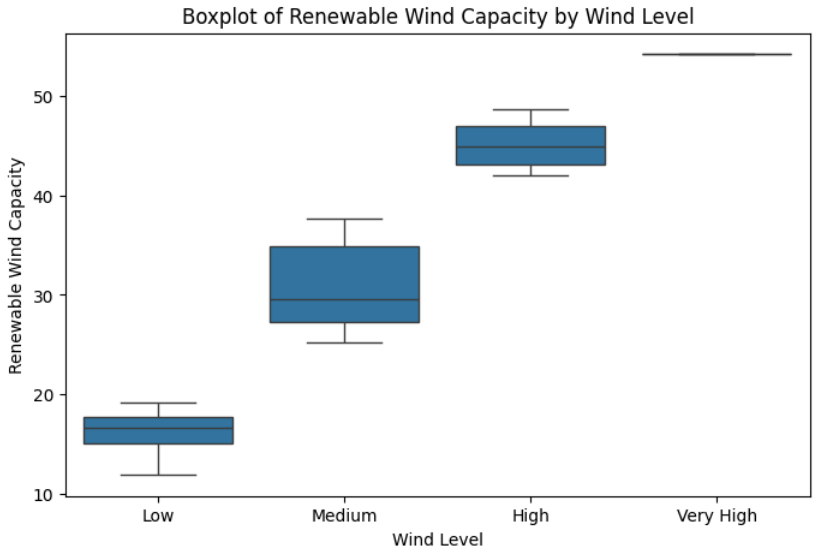


Box Plot

In [ ]:
```
#Renewable Wind Distribution
plt.figure(figsize=(8, 5))

sns.boxplot(
    x='Wind_Level_241UDS0096',
    y='Wind_Capacity_GW',
    data=df_241UDS0096
)

plt.title('Boxplot of Renewable Wind Capacity by Wind Level')
plt.xlabel('Wind Level')
plt.ylabel('Renewable Wind Capacity')
plt.show()
```

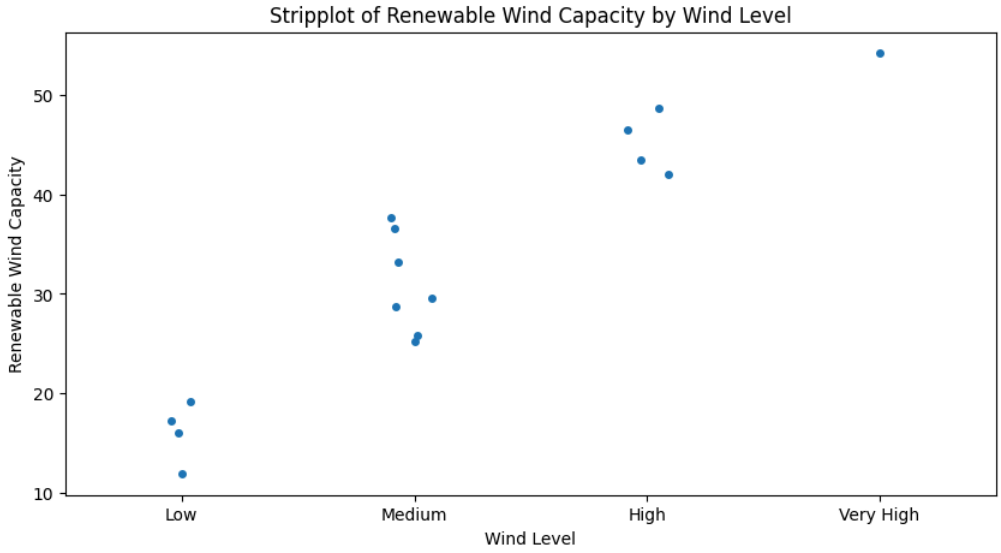


Swarm / Strip Plot

In [ ]:
```
plt.figure(figsize=(10, 5))

sns.stripplot(
    x='Wind_Level_241UDS0096',
    y='Wind_Capacity_GW',
    data=df_241UDS0096,
    jitter=True
)

plt.title('Stripplot of Renewable Wind Capacity by Wind Level')
plt.xlabel('Wind Level')
plt.ylabel('Renewable Wind Capacity')
plt.show()
```

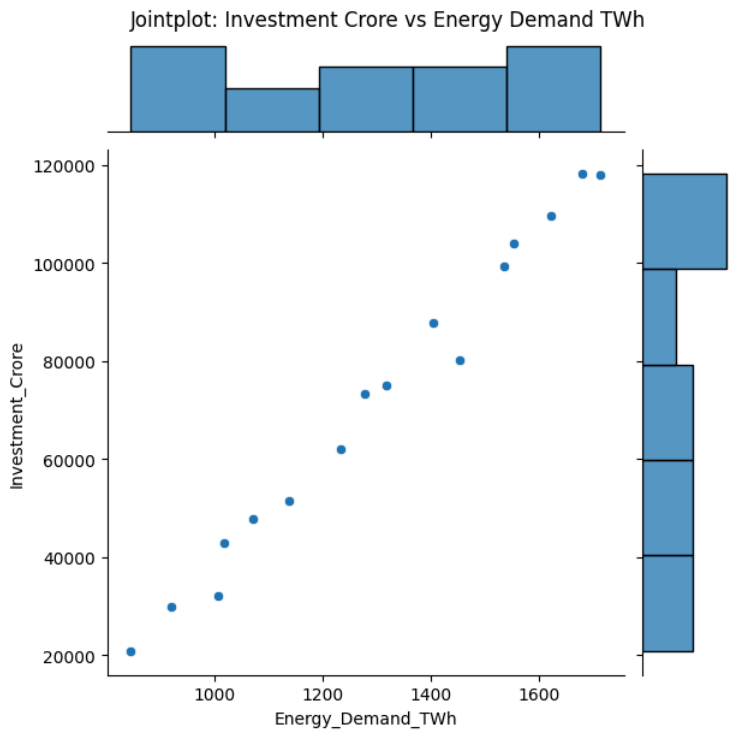


Joint Plot

In [ ]:
```
sns.jointplot(
    x='Energy_Demand_TWh',
    y='Investment_Crore',
    data=df_241UDS0096,
    kind='scatter'
)

plt.suptitle(
    'Jointplot: Investment Crore vs Energy Demand TWh',
    y=1.02
)

plt.show()
```



Heatmap

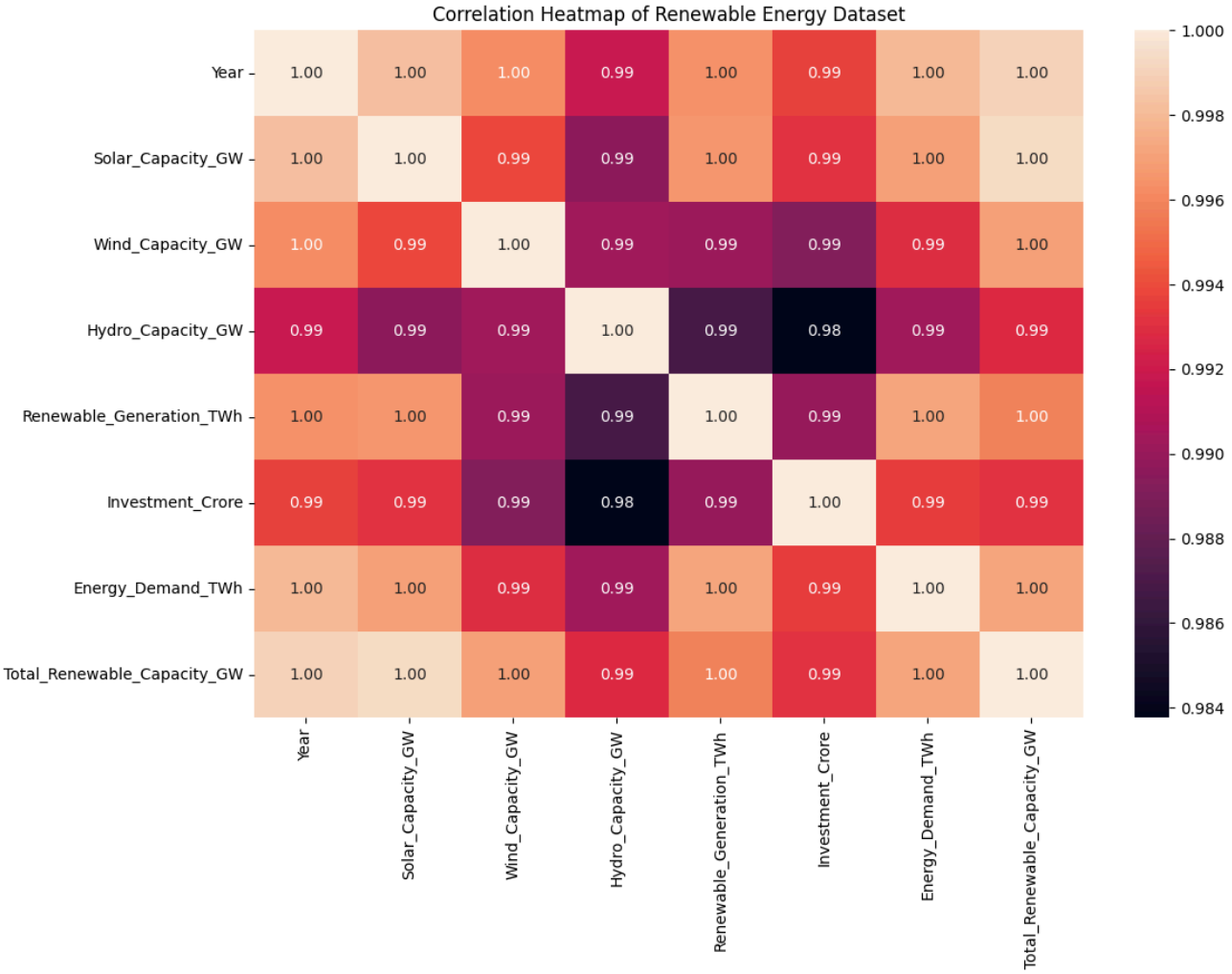
In [ ]:

```
plt.figure(figsize=(12, 8))

correlation_241UDS0096 = df_241UDS0096.corr(numeric_only=True)

sns.heatmap(
    correlation_241UDS0096,
    annot=True,
    fmt='.2f'
)

plt.title('Correlation Heatmap of Renewable Energy Dataset')
plt.show()
```



RESULT:

Renewable energy data was successfully analyzed using data processing and visualization techniques.
The analysis helped identify key trends and relationships among renewable energy capacity, generation, demand, and investment.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.